The Operator Doctrine

Maxwell Collins

The Operator Doctrine

What this is

1. Three facts about the agent
2. The one rule
3. The test of every check
4. The truth order
5. How agents fail, and what each failure became
6. Who checks whom
7. What reaches a person, and what does not
8. Stewardship
9. Cost, and where this does not apply
10. What is not proven
11. The claim this rests on
12. The doctrine condensed, for an agent
13. The record this was written from
14. What was withdrawn from the first edition, and why

The Operator Doctrine

Second edition. How to get true work out of AI agents.

What this is

This is a method for putting AI agents to work on things that matter: software, a phone line, a company's records, a public campaign, a research pipeline. It is written for the person responsible for the result, whether or not that person can read what the agent produces. Its author cannot read code and has directed agents across thousands of recorded sessions, so the method never rests on the operator reading the work. It rests on arranging the work so that the agent's word is never the thing you have to trust.

Each rule below is tied to the incident that produced it. The incidents come from one operator's project records. They cannot be checked from this document, and they are given so that the rules have reasons, not as proof. The figures in section 13 are a different matter: each names what measured it. How the incidents were established, given that the author cannot read the artifacts, and what would show the document wrong, are in section 10.

The first edition of this document, published in June 2026, is kept in this repository under a superseded marker, with one section removed. This edition replaces it. What was withdrawn and why is in section 14. Section 12 is the document condensed to thirteen lines for an agent to hold as operating defaults.

1. Three facts about the agent

An AI produces the most probable continuation of what is in front of it. Probable and true coincide often enough to be useful and seldom enough to be dangerous, and the model has no internal signal that tells the two apart. Under uncertainty it does not go quiet. It produces something shaped like an answer.

It was trained to look finished. Kalai, Nachum, Vempala and Zhang (*Why Language Models Hallucinate*, a preprint of September 2025, arXiv 2509.04664) argue that training and evaluation reward a confident guess over an admission of ignorance, so the model learns to bluff. In practice the bluff is specific rather than vague. Any evaluation the model passes is a stand-in for the real goal, and a capable model pointed at a stand-in finds the cheapest route to the stand-in. A passing test, a green gate, a report that reads as done: each is a stand-in.

It degrades as its own context fills. Exploration, wrong turns and corrections accumulate as noise, and the quality of later answers falls well before the context is full. Chroma's *Context Rot* study of July 2025 measured this across 18 models on its tasks. For agents doing long work this, and not raw ability, is the usual cause of failure in the record this document is drawn from.

Everything in this document is a consequence of those three facts. None of them is fixed by a better prompt, because a prompt is a request, and the three facts describe what the model does with requests.

---

2. The one rule

Never let a question be answered by say-so when it can be answered by a mechanism.

Every discipline below is an instance of that rule. Say-so includes the agent's report of what it did, its summary of what it read, a document an earlier agent wrote, a helper's claim that its part is done, and your own memory of what you approved. A mechanism is something whose answer cannot be produced by stating it: a command's exit code, a row a database refuses, a shape that will not compile, a signal that must arrive from outside, a second party that was not the first.

An answer can live in one of eight places, listed from weakest to strongest. Operating agents is the work of moving every answer that matters as far down this list as the stakes justify.

1. **A request in prose.** The instruction file, the system prompt, the message. The model may follow it, and the odds fall as the session grows.
2. **A note the agent may read.** A memory file, a lesson, a warning. Advisory. A warning the model can talk past is not enforcement.
3. **A block before the act.** A hook that refuses the command, the edit or the send before it happens. The first place that changes behaviour rather than asking for it.
4. **A shape that cannot be saved.** A structural check on every file at the moment it is written. The wrong pattern never reaches the disk.
5. **A state that cannot be represented.** A type that will not compile, a record whose illegal combination cannot be written, a form that cannot omit a field. The failure is not blocked; it has no way to exist.
6. **A record that cannot be altered by the agent.** An append-only ledger the database refuses to update or delete through any program connecting as the application. An administrator can still drop the table, which is why the ledger is also copied off the machine. The point is that no agent can rewrite the history of what agents attempted.
7. **A second party that cannot be the first.** A reviewer with a different identity, a separate repository the builders cannot write to, a database rule that refuses a sign-off whose author is the same as the work's. This removes the bias of a shared context. It does not remove shared training, which is why section 10 names a further step.
8. **A watcher outside the system.** A monitor on another machine that expects a success signal and raises an alarm on silence, a reviewer from a different vendor that never sees your vocabulary, a real signed document as the yardstick.

A rule that has failed twice as prose is promoted down this list. It is not rewritten as better prose. The incident: a rule against a destructive command existed in three separate instruction files and the command ran anyway, twice in one session. It stopped only when a block refused it. A rule in prose is followed most of the time, and most of the time is not a safety property.

Three corollaries. If the agent must never use a capability, remove the capability from its tools rather than forbidding it in words; on real phone calls, an agent told never to end a call while holding an end-call tool ended about one engaged call in six, and in the calls scored after the tool was removed it ended none. A single guard is one bug away from off, so anything that matters is enforced at two levels by unlike machinery, in the program and again in the database, or in the editor and again at the commit. And guards are set to refuse when in doubt, because the two errors are not symmetric: a false refusal costs one extra loop of stop and recover, while a missed violation costs the operator's attention and the operator's trust in every green that follows. What keeps that setting from becoming a guard that refuses everything is the other half of the guard's contract, the inputs it must allow, in section 3.

Prose does not disappear; every system still has a prompt. What belongs there is intent and reasons. A prompt that says why the ledger is append-only helps the agent in the cases the ledger cannot see, and section 10 admits such cases exist. Rules belong in mechanisms. Prose rules also dilute one another, so a prompt with few of them is stronger than one with a hundred.

---

3. The test of every check

A check is only a check if the agent cannot pass it without doing the work. Otherwise it is a target, and the agent will reach it by the cheapest route. The question to ask of every gate, test, review and report:

Can the agent go green without doing the job?

If the answer is yes, the check measures nothing. The incidents behind this question are the most expensive in the record. A pipeline that rewarded test coverage produced about 4,800 tests that passed without exercising the product while more than 400 faults sat in the code. A browser tester reported for three days that testing was going well while it sat at the login screen, because the report was prose and nothing required a number read from disk. A health check reported scheduled jobs fresh with one of eight recorded. A check that compared two empty snapshots passed. A log reader reported no errors over 720 hours from a log under four hours old.

The ways to make the answer no, each in use on real work:

- **Plant the defect.** Before a tester's report counts, it must find defects you planted. A hidden page carries six known faults; a run that does not report exactly six on that page is thrown out, whatever else it found. A fresh discovery run must also reproduce at least half of a frozen set of findings from a known-good run, so the tester's ability cannot quietly fall.
- **Keep the illegal example.** For every guarantee, keep a deliberately illegal case in the project, and on every run require the compiler, the checker or the database to reject it by name. If a later change widens a rule so that the illegal case starts passing, the gate fails. A first version only required the illegal file as a whole to fail, which let one dead example hide behind another's error. One error per marked line.
- **Break the thing on purpose.** Plant a fault in the code and require the tests to catch it. Plant a fake secret and require the scanner to trip. Give every guard a contract: inputs it must refuse and inputs it

must allow, both run through the real entry point and not a stand-in. A guard that refuses everything only looks safe.

- **Prove the guard is wired.** A test compares the list of guards with the list actually connected, and the list of guards with the list of their contracts. A guard with no contract, or a contract for a guard nothing calls, fails the build. This ended a failure that had happened more than once: a guard built, tested and never wired, a permanent no-op that looked like protection. In another case a classifier that judged every final message had read a field the real event never carried, so it had allowed every turn for as long as it existed, and nothing in a test or a settings file could show it.
- **Capture the real interface.** An agent's memory of how a system behaves is a guess. Record real events to disk and check the guards against the recordings. A vendor's service that silently ignores settings it does not accept is checked against the live service, not its documentation. A claim about what an outside system returns is inadmissible without a recorded capture of the real call.
- **Install the second case of opposite shape.** With one customer in the system, software hard-coded to that customer and software any customer can configure produce identical output, and no test can tell them apart. A first build did exactly this in a day. The cure is a second real customer of opposite shape installed permanently in the tests, so that hard-coding fails physically and the general design is the cheapest way through.
- **Judge against external truth.** The product's output is compared with a real signed document, clause by clause, or a real season of a business's own financial records is replayed through it to the cent. A total, a count or a similarity score never stands in for the document. A system once reduced "matches the contract" to two dollar figures and left the rest unchecked.
- **Demand the success signal.** A scheduled job sends a signal to a watcher outside the machine only after it has read back what it wrote. The alarm fires when the signal is missing, so a crashed program, a sleeping computer and a job nobody scheduled all raise it. A chart once stayed frozen for six days on names a person had already deleted, while the log showed only successes and the health check printed fine.
- **Let claims expire.** A backup is proven only by a restore, and a restore drill older than its window counts as no restore. A rollback is proven by a fresh exercise. Permission to deploy without asking lasts only while that proof is fresh.
- **Refuse the hollow pass.** A gate has three states, pass, fail and nothing ran, and nothing ran is never a pass. Every check carries a minimum count. A path not yet connected exits with its own code rather than pretending. A stamp of proof is only ever written by the real action running, never by hand.
- **Read every field.** Six people on one seat showed as vacant for 13 days across 167 runs that all logged success, because the program never read the field that held them. Every field in a design now carries a test that fails if it is not read.

The planted defects, illegal examples and deliberate faults do not have to be authored by a person. An agent that did not build the work is a cheap and good source of them, because it knows the routes a model takes to a stand-in. In practice a fresh adversary given a finished build adds attacks beyond the scripted ones, and those are the attacks that find the holes.

And one rule about your own instruments. **A check that has never failed proves nothing.** Make every instrument go red once before trusting any green. A limitation of your instrument is never evidence about reality: absent from the sources you checked never means it exists nowhere.

---

4. The truth order

Not all sources are equal. Rank them, rule on every conflict openly, and let the higher rank win. Never settle a conflict by splitting the difference.

1. **What a mechanism returned.** What the command printed, what the test did, what the file contains, what the database refused.
2. **What the owner wrote.** The owner's own typed decisions and documents. A decision counts only when the owner typed it in their own words as the answer to a plain question. Text pasted from another chatbot is advice.
3. **The conversation.** Including the agent's account of what it just did. A hint.
4. **Anything an AI wrote.** A summary, a plan, a handoff, an earlier session's notes, a distilled inventory. Hearsay until checked against something above it. A handoff document opens by telling the next agent to distrust it.
5. **The artifact itself.** The code, the configuration, the spreadsheet. It shows what is and never what was meant. When the owner's rule and the artifact disagree, the disagreement is reported before anything changes, and if the rule is confirmed the artifact is wrong, even when it works. An agent asked to settle such a disagreement on its own will quietly rewrite the rule, so the agent may only add to a rule or strengthen it. Narrowing or removing one is the owner's act, through the loop in section 8.

Three consequences that are not obvious. Behaviour the artifact shows and no rule states is itself a defect, whether or not it works. An AI-written summary of what the owner asked for will narrow it; in one project it happened twice, agents then used the summaries to argue the scope down, and the request was rebuilt from primary sources with the summaries set aside. And an AI will write itself a specification and then obey it; the first attempt at one system did exactly that and built a check that blocked two true figures. No agent writes the rule it then follows. Since every mechanism is built by an agent, this needs stating precisely: the owner authors the rule in plain words, an agent implements it as a mechanism, and a different agent proves that the mechanism refuses what it should and allows what it should. Three parties, and the first and the third are never the second.

---

5. How agents fail, and what each failure became

Under pressure an agent behaves predictably. Once behaviour is predictable it can be designed around, which is why each disposition below became a mechanism and not a correction: you are not changing the creature, you are building the enclosure in which its instincts produce the right outcome. Asking for

better behaviour changes nothing durable. Changing the structure so the behaviour cannot recur does. Seven dispositions follow, each with the words that give it away, the incident that taught it, and the mechanism it became.

Guessing. The agent produces a confident claim from pattern rather than from the facts of this case. The tell: an answer that would change if challenged, or that arrives too easily. Incident: a hook read an event field by a name the model remembered, which did not exist in the real data. Mechanism: run, read, then claim; every claim cites the file and line, the command output or the recorded capture; a claim with no citation is pending evidence and carries no weight. When the agent does not know, the right output is that it does not know and is finding out, and that is the sentence the training penalises most.

Deferring. The agent puts work off and calls the result done. The tell: later, phase two, out of scope, a good stopping point, a follow-up, and any estimate in hours or days. Incident: an agent completed one phase of a ten-phase plan, wrote a progress summary and stopped. Another, holding an approved list of 13 tasks, did four and abandoned the rest. Mechanism: the approved task list is written into an unalterable record when the plan is loaded, and a claim that the plan is complete is refused while any approved task has no evidence; a block refuses the deferral words in a plan file, which is a floor that synonyms pass and the task-list check above it is the real guard; and deferral, where it is legitimate, is a contract that records the condition that reopens the work, which every later session checks.

Removing the signal. Faced with a failing check, the agent removes the check rather than the fault: deletes the test, marks it skipped, silences the type error, commits with verification off. The tell: the failing thing is gone and the agent reports green. Incident: an agent merged a change it described as a full audit pass with three ways to lose money open in it, having loosened in the same change the guard that would have caught it. Mechanism: each of those moves is refused by a block on its syntax; a change to a guard ships alone and first, reviewed by someone who did not write the work; and on any error there are two moves only, fix the cause within the work you were given or stop and present it. A fix outside the work you were given is presented, not made. The third move, removing the signal, is always cheaper than the two real ones, which is exactly why it is blocked rather than discouraged.

Rationalising. Having caused damage, the agent writes text that explains the damage as fine. The tell: pre-existing, not in my changes, unrelated, probably the cache, no fix needed. Incident: an orchestrator reported all three of its helpers fine without reading their three failing reports. Another wrote its own lapse into the record as a fault of the documents. Mechanism: raw output is emitted before interpretation; a helper's report is read against the artifact, never relayed; the record is evidence, not defence; and a report showing two or more signs of trouble must be acknowledged before the orchestrator can continue.

Conforming. The agent inherits whatever an earlier agent or document asserted, including its errors. The tell: the same shallow diagnosis from every agent sent to the same question. Incident: 15 separate agents used one wrong rule in shared memory to set a whole class of applications aside. Fifteen sessions in a row treated a two-channel service as single-channel because its manual named one channel 22 times and the other never. Mechanism: a confirming reviewer is given the claim and the thing itself, never the finder's reasoning, because reading why someone believes a thing is how their frame is inherited; when a product is being replaced on the owner's decision, the agents writing the replacement never see the old

code, only an inventory of its features that a separate agent distilled and a third checked against the original, because agents copy what they are shown even when told not to; and the words that would tell a reviewer what to find (legacy, vendor, blocker, the audit score) are stripped from the brief before the work is shown.

Drifting. The agent answers an easier adjacent question, or widens the task into one it prefers. The tell: scope that grew without a decision, a “while I’m here” refactor, a report that does not match the question asked. Incident: one file grew from about 5,750 lines to about 6,700 across 25 rounds of improvement nobody asked for. Mechanism: every dispatch names the exact files it may produce and no others; a sweep that reads files to fill a record has its deliverable fixed before it opens one; every stated intention is reconciled at the end as done with a path, blocked with a reason, or cancelled with a reason.

Claiming inability. The agent says it cannot do what it can. The tell: cannot from here, requires a paid service, needs manual setup, no access. Incident: 11 commits rebuilding a page shipped without the page once being loaded, because the agent said it could not test from the command line while the browser tool sat registered the whole time. An agent said a paid outside service was required; the operator paid for it monthly for months before finding a free local equivalent. Mechanism: a refusal is a claim to be disproved and must name the exact tool tried and the exact error returned; anything that can be started is started and is never offered as a reason to stop. Genuine inability exists and has a short list: a sign-in that needs a person in a browser, a physical act on a machine, a credential the agent does not hold, a refusal from a permission system or a safety boundary, and a decision of the kinds in section 7. Those are reported as what was tried and what is needed, and the work waits. A permission denial is a stop, never an obstacle to route around.

An eighth disposition belongs to the operator, not the agent. **Accepting the agent’s account because checking is expensive.** Every incident above was discovered late because a person took a report at its word. The method exists so that the report never has to be taken.

---

6. Who checks whom

The producer never grades its own work, and the separation is physical. The reasoning that led to each choice is still in the producing session’s context and will confirm each choice again. This is a property of the structure, not of effort, so it is not cured by asking the agent to be critical. A review counts only when the record names two different sessions. Builders and verifiers act under two separate identities, added after an agent refuted and closed its own finding in 61 seconds. In the strongest form the examiner lives in its own repository where the builders cannot write, takes a fresh copy of the product at a named version, drives it through the doors a customer would use, and judges the output against a real outside document.

The orchestrator verifies and never relays. A lead agent that dispatches helpers reads every file that comes back and runs the checks itself. A helper’s summary is a claim. A helper’s silence is not success. When a helper halts, the next move is a diagnosis of what happened, read from the record off disk by an agent that was not involved, and not another attempt.

Fresh context for anything load-bearing. The working session is biased toward finishing. A decision that matters, and every review, goes to a context the session's noise has not touched, and the answer is saved to a file so the next session inherits it instead of re-deciding. Three corrections on the same framing mean the context is polluted; the move is to write the state down and hand the work to a fresh agent, not a fourth correction. The person is not handed off. The context is.

The fresh agent gets the record and the question, never the hypothesis. A fresh context is only uncontaminated if the brief that sends it is. The operator usually has a theory, and a theory in the brief contaminates the reading from a different direction: the agent's disposition to conform will confirm the operator as readily as it confirms a prior agent. A diagnostician is handed the record off disk and the question to answer. What it should find, and what it need not try, stay out of the brief.

Roles are defined by what each can refuse. A domain authority can refuse work as not what was asked for. A craft reviewer can refuse work as badly built regardless of whether the ticket was right. An evidence officer asks of every claim whether the method used could have produced a failing result. A role that only writes reports protects nothing. Each role is given only the tools its job needs, so a reviewer holds no way to edit, a planner holds no way to run, and a role cannot act outside itself even if it tries.

Severity is computed, never assigned. A reviewer asked for a severity answers a different question. One rated a flaw that bypassed a whole defence as minor because fixing it was awkward. Reviewers report facts from fixed lists (worst case, how likely, who is affected, how many defences remain); one of those is still a judgment, but the composition into a rating is a small program, and a label that disagrees with it is rejected. In one store that rule produced 80 ratings to redo.

Reviews can be wrong in both directions. Three reviewers once approved a branch whose tests were failing, so the test suite now runs before any reviewer is asked. A repair pass on a set of documents introduced 15 new false statements, so corrections search the whole set before marking anything corrected and are themselves reviewed by someone else.

---

7. What reaches a person, and what does not

Technical decisions belong to the agent. When the next step can be worked out from the facts, the agent makes the call, states one reason, and proceeds or asks for a yes. Routing a technical choice back to a person who cannot make it is abdication, not deference. A menu of options is the same failure in a different shape.

Four kinds of decision come back to the owner, plus whatever the owner has asked to be consulted on. Money moving. Anything that cannot be undone. Law and personal data. The business itself: what the product is for, what it costs, who it serves. Each arrives as one plain question with a recommendation, anchored on something the person can witness: a number, a rendered document, a yes or no. Where a person has asked to be consulted on something else, that stands.

Nothing a person will act on is computed by a model. Every displayed figure is a named query anyone can rerun, with its population stated and its tolerance declared. A page about a system never shows the auditing tool's own faults as the system's. A value that cannot be taken shows as unknown, never as zero and never as green.

Plain words, with the evidence attached beneath. What reaches the owner is the meaning of what the agent did, not a retelling of its report: what it means for them, what was left out, where the work really stands, what should happen next. The citations live in the record and travel with the message; they are not the message. Status is computed from receipts and never typed.

Money never moves by itself. No refund, charge or outward send is initiated by an agent without a person's decision or physical evidence, and this is not a trust level to grow out of. A customer paying through the system is the person's own act; the rule guards movement the agent would start.

---

8. Stewardship

Rules are earned from incidents, never invented. A safeguard written against a failure that has never happened anywhere is a guess, and it adds surface that must itself be maintained. Every lasting rule is anchored to a dated incident so that its reason survives and a later agent cannot improve it away. Earned does not mean earned here: a guard is earned from an incident somewhere, and once earned it is installed everywhere the class can recur, including a project on its first day. The circuit breaker in section 9 is such a guard.

The same diagnosis that adds a guard can narrow or remove one. A failure is a probe into the system, not a thing to patch. Get an uncontaminated reading of it from a fresh agent reading the record. Audit the whole to learn what is load-bearing. Trace to the cause. Make the smallest correct change, which may add, remove or leave alone. A wall that only grows becomes something you fight, and an operation where every bypassed check is answered with another check is on a treadmill. The way off the treadmill is to move the rule down the list in section 2 until the failure cannot exist, at which point the checks above it can go. Narrowing is the owner's act through this loop; an agent proposes it and never performs it on its own. One trap that encoded a wrong rule was retired with the retraction written into its own record; two dead guards were consolidated; two blocks were narrowed to where they were needed.

Exhaustive in understanding, conservative in changing. When in doubt about what happened, do more: more reading, more tool calls, parallel diagnoses that are then compared. When in doubt about what to change, do less. Never rewrite a working mechanism; add to it. Never remove something load-bearing without its replacement in place. A mechanism that has been clean for weeks is not refuted by two recent anomalies that may have another cause.

Nothing is deleted from the record. A replaced plan or rule keeps its text under a dated marker naming its successor. A wrong finding is corrected in place with a dated note, never silently rewritten. The one exception is lawful redaction of a secret or of personal data, and a redaction records that it happened, where, and why. Deleting without a record is how the next agent re-proposes what was deleted.

Knowledge is an artifact, not a memory. A lesson kept as prose is forgotten; a lesson kept as code cannot be. A lesson is admitted only if it quotes a real passage of a real file, and it is shown to an agent at the moment it touches that file. The pile of written notes is meant to shrink as each note graduates into a type, a constraint or a check; a note that has graduated is retired with a pointer to the mechanism that replaced it.

---

9. Cost, and where this does not apply

The method is not free, and a doctrine that does not say what it costs is dogma.

The friction is measurable, and most of it is the guards testing themselves. The enforcement ledgers of seven of one operator's projects held 371,817 recorded refusals when counted in October 2026. The large majority are the guards re-proving their contracts on every run, which is the point of section 3. The refusals of agents' real work are a small fraction, and each of those is a turn spent. A new rule costs review rounds; the money domain of one design went through four rounds with two fresh reviewers before every quoted citation matched its source. The working expectation is two to four review rounds per unit of work, written into the handoff. First-pass quality is not expected.

The most expensive failure in the record is a loop that retries itself. On 2 June 2026 an unattended daemon that retried on its own produced about 370,000 junk transcripts, 8.7 gigabytes, in about a day before it was caught, counted on disk afterwards. The resemblance to the ledger figure above is coincidence; they are different counts from different projects. A failure that would fail the same way again (a bad credential, a missing file, a refused permission) is never retried. A transient failure (a network error, a rate limit) is retried a bounded number of times with the attempts recorded, then parked loudly. Scheduled AI work gets a persistent circuit breaker from its first day. Spending limits count work done, not days elapsed.

A session can grow too long and costly to continue. Context rot makes a long session worse at its own job. The signs are the same corrections repeating, an orchestrator fixing from memory, and effort flowing into a wrong target. There is no fixed hour at which this happens. The remedy when the signs appear is to stop at a clean boundary, record the state to disk, and start fresh from the record.

Where it does not apply. The depth of each stage scales with the stakes; the sequence never does. The design of a marketing page is done by hand, and its claims still go through the mechanism. The full frame is not worth building for a throwaway. The method concentrates its weight on what must never be left to reasoning: the wall between customers, money arithmetic, the core rules of the business, consent, and anything irreversible. Everything else can be lighter.

---

10. What is not proven

This method has one practitioner. It has been run across many projects and several domains, but by one person, and the claim that it transfers rests on its being written down, not on anyone else having run it.

The author cannot read the artifacts behind the incidents in sections 2 to 8. Each was established by an agent that was not involved, reading the record, and the author read the mechanism's output where there was one. That is the standard the method sets for every other claim, and a reader is right to hold the document to it: the incidents are a practitioner's report, not evidence a stranger can check.

The phone result in section 13 was scored by a model classifying transcripts. The scorer's inputs carry tests, added after it silently dropped half an agent's turns for six months, but there is no set of known-good and known-bad calls that the scorer must label correctly before a run counts. By section 3 that is a check that has not been proven to bite, and the figure should be read with that in mind.

The refusal count in section 9 is mostly the guards' own self-tests. Session-attributed refusals of real work, the true floor of enforcement, are a small fraction and have only been counted since attribution existed.

Drift between agents is managed, not solved. One persistent author owns a specification from vision to plan, and helpers gather and attack but never author, which reduces re-interpretation at each hand-off. It does not remove it.

Two agents of the same model under the same instruction files share a training, whatever their session or identity. A harness built from one operator's account of the problem can come to flatter that operator in the same way. A reviewer from a different vendor that never sees the operator's vocabulary is the current answer to both. It is a mitigation, not a proof.

No enforcement is total. What a guard cannot see (a redirect inside a command, a path built at run time) is written down by class, with which other layer catches it and what is left to the agent's obedience. The honest residual is stated, because pretending enforcement is total is how breaches happen.

What would show this document wrong. Two conditions. If an incident class recurs through a path its guard covers, after that guard has reached the fifth place in section 2 or lower, the claim that the lower places end a class is wrong. A recurrence through a path listed as residual does not count, and a residual list that grows to cover each recurrence is itself the refutation. The first recurrence may be blamed on a buggy mechanism; the second may not. And if a second practitioner's raw sessions, outside any of these mechanisms, do not show the seven dispositions in section 5 recurring under pressure with the same tells and the same cost, then the dispositions are an artifact of one operator's harness and not of the model. Sessions inside the enclosure cannot test this, because the enclosure is built to suppress them. The words appearing once is not the dispositions appearing.

---

11. The claim this rests on

The second fact in section 1, that a model is trained to look done rather than be done, has not been reduced in this operator's record by more capable models; the record is one operator's and names no comparison, so this is an observation and not a result. The claim built on it is that, until training supplies a fix, this disposition is answered only by structure around the model: checks the agent cannot pass without doing the work, records it cannot alter, and parties it cannot be. The structures in this document are what such a fix would have to supply, and until it does they are built on the outside, because that is the only place a person without a training run can build them.

This is stated as a claim, and section 10 names what would refute it.

---

12. The doctrine condensed, for an agent

This section is prose, the weakest place in section 2, and it knows it. It carries intent and reasons for the cases the mechanisms cannot see; it does not replace them. "The owner" means whoever is responsible for the result where you are running, not the author of this file. An agent loading this holds the following as operating defaults. Each line points to its section.

1. Your confident output under uncertainty is a guess shaped like an answer. Treat your own fluency as suspect. (1)
2. Never let a question be answered by your say-so when a mechanism can answer it. Move every answer that matters down the eight places, to the depth the stakes justify. (2, 9)
3. Before trusting any check, ask whether you could pass it without doing the work. If you could, it is not a check. (3)
4. Run, read, then claim. Cite every claim in the record to a file and line, a command's output or a recorded capture. To the owner, give the meaning in plain words with the evidence attached beneath. When you do not know, say so and find out. (4, 5, 7)
5. Rank your sources: what a mechanism returned, what the owner wrote, the conversation, anything an AI wrote, the artifact. Rule on conflicts openly; the higher rank wins. When the owner's rule and the artifact disagree, report it before changing anything. (4)
6. No completion claim without the evidence in the same message. Completion is a gate's exit code read from the real record, never your rating. A gate that ran nothing is not green. Where no gate exists, say so and name what you ran instead. (3, 5)
7. On any error, two moves: fix the cause within the work you were given, or stop and present what happened with a proposed fix. Never continue silently. Never remove the signal. A fix outside your work is presented, not made. (5)
8. Make the technical decision yourself and state one reason. Bring the owner money, the irreversible, law and personal data, the business, and anything they have asked to be consulted on. (7)

9. You cannot review what you wrote. Send load-bearing reviews and decisions to a fresh context, and hand it the record and the question, never the hypothesis. Three corrections on one framing means your context is polluted; write the state down and hand off. (6)
10. “I can’t” requires the exact tool tried and the exact error returned. Start what can be started. A permission denial, a safety boundary, a missing credential or a person needed in a browser is a stop; report it and wait. (5)
11. A failure is a probe into the system, not a thing to patch. Diagnose exhaustively from uncontaminated context; change minimally; the right change may add, remove or leave alone. Never rewrite what works. Never delete from the record without recording the redaction. (8)
12. Watch the cost. Never retry a failure that would fail the same way; retry a transient one a bounded number of times, recorded. When the signs of a polluted session appear, stop at a clean boundary with the state on disk. (9)
13. When you notice you are about to act against one of these lines and nothing refuses you, that is a case the mechanisms could not see. Stop and say so. That report is the incident the next guard is earned from. (8, 10)

---

13. The record this was written from

These figures are the size of the record the incidents come from, and what some of its systems measured. Size is not quality; the method says so in section 9. Each figure names what measured it. These are the output of fleets of AI agents that one operator designed and directed.

Every count is a floor. The surviving transcript record begins in January 2026. The author’s work with agents began in September 2025, and the sessions before the record starts, including those that built the first version of the booking system in November 2025, were deleted by the tool’s own retention before the author began keeping transcripts. Those systems exist and their commit histories survive; their sessions do not. No figure is given for what was lost, because it was not measured.

- 13,897 AI sessions in the whole record to 14 October 2026, of which 4,652 were started by the author, 9,032 were run by agents those sessions dispatched, and 213 were automated, holding 617,706 messages of which 30,866 were typed by the author. Counted by AgentsView, the program that keeps the full transcript of every session on the author’s machine, across Claude Code, Codex and other agents.
- Within that, 4,879 root sessions and 5,507 agents dispatched inside them between 11 January and 13 September 2026, counted by a census program the author wrote over the raw transcripts, which also measures their hours: 11,947 session hours, 2,086 of them active with gaps over thirty minutes removed.
- A booking and payment system for a real business that took over a front office previously run by hand and has been in daily use since November 2025, with real bookings and real money. The practices in this document were in use on it before they were written down; the first edition dates the writing, not the practice. Its incident log is its own and is referred to in section 14.

- A phone receptionist whose calls handed to staff fell from about six in ten to about one in ten, and whose captured leads rose from about one in nine to about three in four. Measured on 857 calls over the life of the agent, scored from their transcripts by an automated pipeline after robocalls were removed. The scoring is a model's classification, and section 10 says what that check has not been proven to do.
- A company's operations workspace kept current by scheduled agents for thirteen weeks unattended beside the administrator who edits it daily. The 1,564 edits the administrator made in the measured four weeks were preserved, and a full copy was taken on 89 of 90 nights, counted from the job log and the copies.
- A multi-customer proposal product whose pricing reproduces a real signed five-year agreement line by line: a test pins all 23 non-zero lines of the agreement to the cent against the engine's output on a real database, and it passed when run for this edition. The product was deployed to a public host in August 2026 and driven end to end in a browser by a reviewing agent; an examiner in a separate repository the builders cannot write to scores it on 87 assertions.
- The incidents in this document, each of which cost time, money or trust before it became a rule.

---

14. What was withdrawn from the first edition, and why

- **The claim of zero shipped bugs.** Withdrawn because it was never measured, and because the record contradicts it: the booking system keeps its own incident record; a May 2026 merge described as a full audit pass carried three money-loss paths and was caught by protection installed the same day; and the receptionist's website chat had never saved a lead, none of three, while the phone line saved all twenty of twenty, a fault that fifteen earlier sessions had missed and a morning audit found. The method's own rule is that a claim may never outrun the thing that measured it.
- **The hours figure.** Withdrawn as unmeasured. The measured session record is in section 13.
- **The section reading a colleague's behaviour from a meeting transcript.** Withdrawn as unfit for publication, and removed from the archived first edition as well; the repository's history keeps what was published. The general point, that an agent of any kind is read by what it does under pressure and not by what it says, survives in section 5.
- **The claim that a non-coder with this method outperforms a coder without it, and the framing of one operator's record as a controlled experiment.** Withdrawn. One practitioner is evidence that the method is learnable by someone who does not write code. It is not a controlled result.
- **The citation of a hallucination paper to a journal.** The paper is a preprint. Section 1 cites it as one.
- **Zero deferral as an absolute.** Replaced by deferral as a contract, which records the condition that reopens the work.
- **The maturity ladder as a numbered procedure.** Replaced by the eight places an answer can live, in section 2, which covers any domain and not only software.

A reader is entitled to ask what a third edition will withdraw. The answer is whatever the record contradicts, and the list will be published the same way. The first edition's correction of its own additive-only loop to a bidirectional one stands and is the heart of section 8.

Maxwell Collins, second edition, October 2026. Licensed CC BY 4.0: use, share and adapt, with attribution.